Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

1 Overview of Contributions and Artifacts

1.1 Paper's Main Contributions

The main observations are as follows:

C_1 Memory faults are more problematic than computational faults. The underlying reason is the fault origin and propagation determined by unique computation pattern of LLMs.

C_2 Generative tasks are more vulnerable than multiple-choice ones due to the error propagation in sequential generated tokens, especially for reasoning tasks.

C_3 Fine-tuned LLMs for generative tasks are more reliable under memory faults, possibly because the training process enhances their ability to keep the sentence structure and fluency.

C_4 MoE models are more vulnerable than dense models on multiple-choice tasks since the change of expert selection may leads to extra accuracy degradation. However, they are more reliable on generative tasks due to the lower possibility to use faulty expert in the following generation iterations.

C_5 Beam search is more reliable than greedy search for generative tasks under computational faults because it can avoid corrupted tokens by altering to another potential token sequence.

C_6 Using Chain-of-Thought increases the reliability on reasoning tasks because the model can recover from the corrupted tokens in the reasoning process.

1.2 Computational Artifacts

The contributions (observations) are related to several evaluation scripts, which are the computational artifacts as follows. The source code is available at https://github.com/pipijing13/sc25-LLM-reliability-assessment. There are 4 folders, folder "basic" is related to A1 and A3, folder "moe" is related to A2, folder "data" contains all data to draw the figures, and folder "figs" contains all corresponding figures in our paper. The DOI is https://doi.org/10.5281/zenodo.16946564.

A_1 **Overall Resilience Characterization.**

A_2 **MoE evaluation.**

A_3 **CoT evaluation.**

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1 - C_3 , C_5	Figures 3-4, 8, 15
A_2	C_4	Figures 11
A_3	C_6	Figures 16

2 Artifact Identification

2.1 Artifact 1

2.1.1 Relation to Contributions. Artifact 1 is related to Contribution 1, 2, 3, 5. Overall resilience evaluation includes fault injection experiments among all tasks, datasets, and fault models considered in this study. Besides, it also contains comparison of different generation settings for generative tasks(*multichoiceFI.py, gsm8kFI.py, squadFI.py, xlsumFI.py and wmtFI.py*). Each file is related to different datasets. All files are in the "basic" folder.

2.1.2 Expected Results.

- Memory faults are more problematic than computational faults.
- Generative tasks are more vulnerable than multiple-choice ones.
- Fine-tuned LLMs for generative tasks are more reliable under memory faults.
- Beam search is more reliable than greedy search for generative tasks under computational faults.

2.1.3 Expected Reproduction Time (in minutes). For all evaluation experiments, the expected artifact execution time is around 3000 hours on an NVIDIA A100 GPU, which is 180000 minutes. Since the reliability assessment relies on sufficient random sampling, decreasing the num_trials could lead to unprecise results. You may only test the availability and functionality if the reproduction is not doable, where num_trials can be set to 1 or 2. The total setup time is mostly related to model download speed from huggingface, which is around 30 minutes for us. The artifact analysis time is less than 10 minutes.

2.1.4 Artifact Setup.

Hardware. All experiments were conducted across two hardware configurations: an AMD EPYC 7742 64-Core CPU paired with an NVIDIA A100 GPU (Ampere architecture) running Rocky Linux 8.10, and an NVIDIA GH200 Grace Hopper Superchip that combines a NVIDIA Grace CPU with 72 Arm Neoverse V2 cores and an NVIDIA H100 GPU (Hopper architecture) running Rocky Linux 9.3. The difference of fault injection results between these two GPU generations is negligible (<0.03% for raw model performance). The experiments require at least 32GB host memory and 40GB GPU memory.

Software. Cuda 11.7 and PyTorch 2.5. Pytorch can be installed by the following command

```
python -m pip install torch==2.5.0+cu118 torchvision
torchaudio --index-url https://download.pytorch.org
/whl/cu118
```

To install lmeval, please refer to https://github.com/EleutherAI/lm-evaluation-harness. Other required packages are shown in *requirements.txt*.

Datasets & Models. The models and datasets are from huggingface, which are shown in Table 1. The first time you execute each Python file, it will automatically download the datasets and models.

Installation & Deployment. There is no need for compilation. Just simply execute each file in the "basic" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper. Results for different experiment settings can be obtained by

Table 1: Selected LLM workloads.

Tasks	Datasets	Metrics	Test models
General model understanding and reasoning	MMLU AI2_ARC TruthfulQA WinoGrande HellaSwag	Accuracy	Llama3.1-8B Qwen2.5-7B Falcon3-7B
Math	GSM8k		Qwen2.5-7B Falcon3-7B
Translation	WMT16	BLEU chrF++	Qwen2.5-7B Llama2-7B ALMA-7B
Summarization	XLSum	Rouge-1 Rogue-L	Llama3.1-8B Qwen2.5-7B Summarizer
Question Answering	SQuAD v2	Exact Match F1 score	Llama3.1-8B Qwen2.5-7B Falcon3-7B

changing command line arguments (models, datasets, fault models, and generation settings). Note that a double dash is needed for each argument.

To obtain all results, you should test every model and dataset in Table 2. You need access for Llama models on huggingface.Additionally, you should test all 3 fault modes.

For *multichoiceFI.py*, you may use the following command as an example:

```
python multichoiceFI.py --fault_mode neuron --model
qwen --num_trials 1000
```

For *gsm8kFI.py*, you may use the following command as an example:

```
python gsm8kFI.py --fault_mode neuron --model qwen
--num\_trials 1000
```

For *squadFI.py*, you may use the following command as an example:

```
python squadFI.py --fault_mode neuron --model qwen
--num_trials 500
```

For *xlsumFI.py*, you may use the following command as an example:

```
python xlsumFI.py --fault_mode neuron -model qwen
--generation_mode greedy --num_trials 500
```

For *wmtFI.py*, you may use the following command as an example:

```
python wmtFI.py --fault_mode neuron --model qwen
--generation_mode greedy --num_trials 500
```

2.1.5 Artifact Evaluation. Just simply execute each file in the "basic" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using the equation in our paper. Normalized performance = Average bit flip accuracy / Baseline accuracy, you may find these two metrics in the execution output. Results for different experiment

Table 2: Workloads for each file

Files	Datasets	Test models
multichoiceFI.py	MMLU AI2_ARC TruthfulQA WinoGrande HellaSwag	Llama3.1-8B Qwen2.5-7B Falcon3-7B
gsm8kFI.py	GSM8k	Qwen2.5-7B Falcon3-7B
wmtFI.py	WMT16	Qwen2.5-7B Llama2-7B ALMA-7B
xlsumFI.py	XLSum	Llama3.1-8B Qwen2.5-7B Summarizer
squadFI.py	SQuAD v2	Llama3.1-8B Qwen2.5-7B Falcon3-7B
moearcFI.py	AI2_ARC	Llama-3.2-8X3B-MOE -Instruct-18.4B
moemmlu.py	MMLU	
moewmt.py	WMT16	
moeaquad.py	SQuAD v2	
densearcFI.py	AI2_ARC	Llama- 3.2-3B-Instruct
densemmlu.py	MMLU	
densewmt.py	WMT16	
denseaquad.py	SQuAD v2	
nocotFI.py	GSM8k	Qwen2.5-7B Falcon3-7B

settings can be obtained by changing command line arguments (models, fault model, and generation settings). We run 500 trials for translation, summarization, and question answering tasks, while 1000 trials for other tasks. You may ignore all warnings.

2.1.6 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the number of the corresponding figure.

2.2 Artifact 2

2.2.1 Relation to Contributions. Artifact 2 is related to Contribution 4. We evaluate the overall reliability of MoE models (*moearcFI.py, moemmluFI.py, moewmtFI.py and moesquadFI.py*). The code for the dense model is in *densearcFI.py, densemmlu.py, densewmtFI.py and densesquadFI.py*. All the files are in the "moe" folder.

2.2.2 Expected Results.

- MoE models are more vulnerable than dense models on multiple-choice tasks.
- MoE models are more reliable on generative tasks.

2.2.3 Expected Reproduction Time (in minutes). For all evaluation experiments, the expected artifact execution time is around 150

hours on an NVIDIA A100 GPU, which is 9000 minutes. The total setup time is mostly related to model download speed from huggingface, which is around 10 minutes for us. The artifact analysis time is less than 2 minutes.

2.2.4 Artifact Setup.

Hardware. Same as Artifact 1.

Software. Same as Artifact 1.

Datasets & Models. The models and datasets are from huggingface, we evaluate Llama-3.2-8X3B-MOE-Instruct-18.4B model on MMLU, AI2_ARC, WMT16, and SQuAD v2 datasets since they include diverse content. And we compare the results with Llama-3.2-3B-Instruct model.

Installation & Deployment. For overall reliability assessment, simply execute each file in the "moe" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper. To obtain all results, you should test every model and dataset in Table 2. You need access for Llama models on huggingface.

For both moe and dense model, each file is for different dataset, you may use the following command as an example:

```
python moemmluFI.py --num_trials 1000
```

2.2.5 Artifact Evaluation. For overall reliability assessment, simply execute each file in the "moe" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using the equation in our paper. Normalized performance = Average bit flip accuracy / Baseline accuracy, you may find these two metrics in the execution output. Each file is for different dataset. We run 500 trials for translation, summarization, and question answering tasks, while 1000 trials for other tasks. You may ignore all warnings.

2.2.6 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the number of the corresponding figure.

2.3 Artifact 3

2.3.1 Relation to Contributions. Artifact 3 is related to Contribution 6. This artifact contains reliability assessment for reasoning tasks without CoT (*nocotFI.py*). The code with CoT is in *gsm8kFI.py* from the "basic" folder.

2.3.2 Expected Results. Using Chain-of-Thought increases the reliability on reasoning tasks.

2.3.3 Expected Reproduction Time (in minutes). For all evaluation experiments, the expected artifact execution time is around 200 hours on an NVIDIA A100 GPU, which is 12000 minutes. The total setup time is mostly related to model download speed from huggingface, which is around 10 minutes for us. The artifact analysis time is less than 2 minutes.

2.3.4 Artifact Setup.

Hardware. Same as Artifact 1.

Software. Same as Artifact 1.

Datasets & Models. The models and datasets are from huggingface, we evaluate Falcon3-7B-Instruct and Qwen2.5-7B-Instruct model on GSM8k dataset.

2.3.5 Artifact Evaluation. For overall reliability assessment, simply execute *nocotFI.py and basic/gsm8kFI.py* to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using the equation in our paper. Normalized performance = Average bit flip accuracy / Baseline accuracy, you may find these two metrics in the execution output. We run 500 trials for translation, summarization, and question answering tasks, while 1000 trials for other tasks. You may ignore all warnings.

Installation & Deployment. For overall reliability assessment, simply execute *nocotFI.py and basic/gsm8kFI.py* to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper. To obtain all results, you should test every model and dataset in Table 2.Additionally, you should test all 3 fault modes.

You may use the following command as an example:

```
python nocotFI.py --fault_mode neuron --model qwen
--num_trials 1000
```

2.3.6 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the number of the corresponding figure.

Artifact Evaluation (AE)

3 Artifact Evaluation

3.1 Configuration

Parameters belong to the Python files for all artifacts. Please refer to the details in corresponding artifact sections.

- **`--fault_mode`** (default: `weight`): Specifies the fault injection mode
  - `weight`: Inject faults into model weights
  - `neuron`: Inject double-bit faults into neuron outputs
  - `single`: Inject single-bit faults into neuron outputs
- **`--generation_mode`** (default: `greedy`): Text generation strategy
  - `beam`: Use beam search for generation
  - `greedy`: Use greedy decoding for generation
- **`--model`**: Target model for fault injection
  - `alma`: ALMA-7B model
  - `qwen`: Qwen-7B model
  - `llama2`: LLaMA2-7B model
  - `llama3`: Llama-3.1-8B model
  - `falcon`: Falcon3-7B model
  - `summarizer`: Llama-3.1-8B-Summarizer model
- **`--task`** (default: `weight`): Target dataset for fault injection
  - `mmlu`: MMLU
  - `arc`: AI2_ARC
  - `hella`: HellaSwag
  - `wino`: WinoGrande
  - `truth`: TruthfulQA

- **`--num_trials`** (default: `500/1000`): Number of bit flip experiments to perform per input sample. Since the reliability assessment relies on sufficient random sampling, decreasing the num_trials could lead to unprecise results. You may only test the availability and functionality if the reproduction is not doable, where num_trials can be set to 1 or 2.

3.2 Artifact 1

3.2.1 Artifact Setup.

Hardware. NVIDIA A100 GPU or NVIDIA H100 GPU. Other NVIDIA GPUs with GPU memory larger than 20GB should also be fine. The difference of fault injection results between these two GPU generations is negligible (<0.03% for raw model performance). The experiments require at least 32GB host memory and 40GB GPU memory.

Software. Cuda 11.7 and PyTorch 2.5, other required packages are shown in *requirements.txt*

Datasets & Models. The models and datasets are from huggingface, which are shown in Table 1.

3.2.2 Artifact Execution. Just simply execute each file in the "basic" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper. Results for different experiment settings can be obtained by changing command line arguments (models, datasets, fault models, and generation settings). Note that a double dash is needed for each argument.

To obtain all results, you should test every model and dataset in Table 2. Additionally, you should test all 3 fault modes.

For *multichoiceFI.py*, you may use the following command as an example:

```
python multichoiceFI.py --fault_mode neuron --model
qwen --num_trials 1000
```

For *gsm8kFI.py*, you may use the following command as an example:

```
python gsm8kFI.py --fault_mode neuron --model qwen
--num\_trials 1000
```

For *squadFI.py*, you may use the following command as an example:

```
python squadFI.py --fault_mode neuron --model qwen
--num_trials 500
```

For *xlsumFI.py*, you may use the following command as an example:

```
python xlsumFI.py --fault_mode neuron -model qwen
--generation_mode greedy --num_trials 500
```

For *wmtFI.py*, you may use the following command as an example:

```
python wmtFI.py --fault_mode neuron --model qwen
--generation_mode greedy --num_trials 500
```

3.2.3 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the corresponding figure.

3.3 Artifact 2

3.3.1 Artifact Setup.

Hardware. NVIDIA A100 GPU or NVIDIA H100 GPU. Other NVIDIA GPUs with GPU memory larger than 40GB should also be fine.

Software. Same as Artifact 1.

Datasets & Models. The models and datasets are from huggingface, we evaluate Llama-3.2-8X3B-MOE-Instruct-18.4B model on MMLU, AI2_ARC, WMT16, and SQuAD v2 datasets since they include diverse content. And we compare the results with Llama-3.2-3B-Instruct model.

3.3.2 Artifact Execution. For overall reliability assessment, simply execute each file in the "moe" folder to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper.To obtain all results, you should test every model and dataset in Table 2. You need access for Llama models on huggingface.

For both moe and dense model, each file is for different dataset, you may use the following command as an example:

```
python moemmluFI.py --num_trials 1000
```

3.3.3 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the corresponding figure.

3.4 Artifact 3

3.4.1 Artifact Setup.

Hardware. Same as Artifact 1.

Software. Same as Artifact 1.

Datasets & Models. The models and datasets are from huggingface, we evaluate Falcon3-7B-Instruct and Qwen2.5-7B-Instruct model on GSM8k dataset.

3.4.2 Artifact Execution. For overall reliability assessment, simply execute *nocotFI.py and basic/gsm8kFI.py* to obtain raw model performance for baseline and fault injected outputs. Then calculate the normalized model performance using equation 1 in our paper. You should test all models in Table 2. Additionally, you should test all 3 fault modes.

You may use the following command as an example:

```
python nocotFI.py --fault_mode neuron --model qwen
--num_trials 1000
```

3.4.3 Artifact Analysis. Scripts for plotting figures are in *figs.ipynb*. The first line of each cell indicates the corresponding figure.